

COSC 1030 Physical Computing Project 1: Blink and Traffic Light

Background and motivation: Traditionally, the style of programming that students are first introduced to is called “sequential programming”. In a sequential program, the instructions are run one at a time. For example in the pizza ordering program that we wrote in COSC 1010, the program asks the user for the size of the pizza, reads in the user’s response and then does something with that information. The program then moves to the next section to ask the user for the crust type. The program proceeds from the top of the code to the bottom. In the real world, programs are more often run in parallel, with many processes happening simultaneously. For example, in GUI driven programs, a button press or a mouse movement will trigger appropriate code. Physical computing is another example where it will be advantageous to have multiple processes running simultaneously. For example, if a temperature sensor registers that the temperature has increased above a threshold, we might want to turn on a fan. If the temperature drops below a threshold, turn on the heater. This kind of event driven programming is more and more common. The Raspberry Pi combined with some introductory electronics is going to be the way that you can continue to grow your knowledge and intuition about parallel computing.

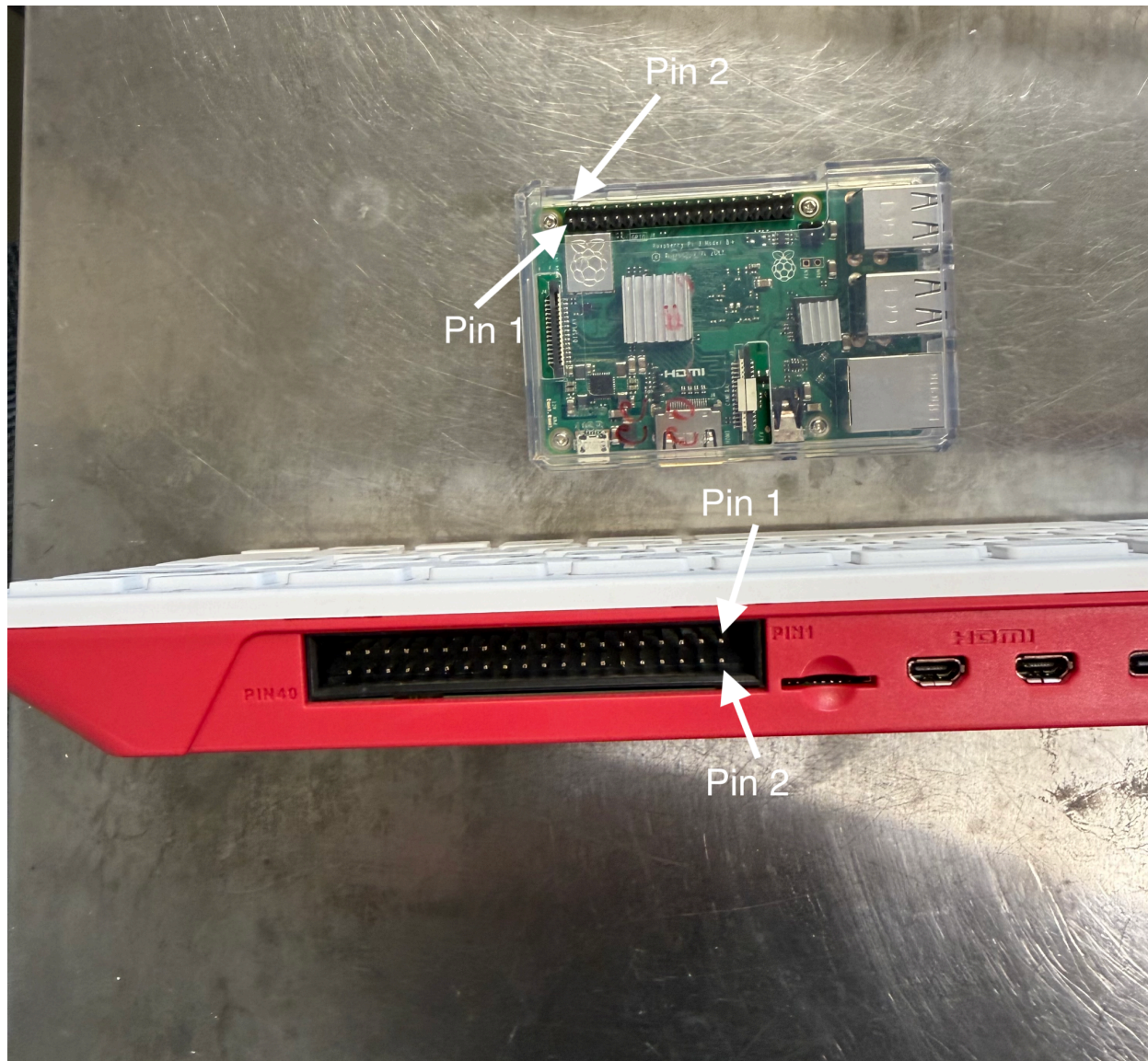
In this assignment, we will learn to use the Python `gpiozero` library to program a blinking light. Note that in the [blink project from the materials in week 1](#), we used a different python library (also called a “python package”) to access the pins (`RPi.GPIO`). The `gpiozero` package is a little more pythonic and may be easier to use.

Equipment Needed:

- Raspberry Pi
- 80 ohm - 1K ohm resistor x 3
- Red, green, yellow LED
- Breadboard
- Jumper wires

The pins on the Raspberry Pi:

Find the pins on your Raspberry Pi. Two models are shown below. On the Raspberry Pi 400 (the one with the keyboard), the pins are on the back side. You should be able to see the words “PIN1” and “PIN 40”. For the 400, PIN 2 will be right below PIN 1, the pin directly to the left of PIN 1 is PIN 3. So all the odd numbered pins are on the top, all the even numbered pins are on the bottom. The Raspberry Pi 3B+ shown in the picture also has 40 pins. Pin 1 on this Raspberry Pi will be the one on the bottom left. Pin 40 will be the top right. As with the 400, on the 3B+ all the odd numbered pins are on the bottom row, all the even pins on the top. These pin numbers are the BOARD numbers or the physical number for the pins. There are several other ways that pins are referred to, but the other one that we are going to talk about is BCM or Broadcom description. The Broadcom description tells more about what the pin does, rather than where it is located.

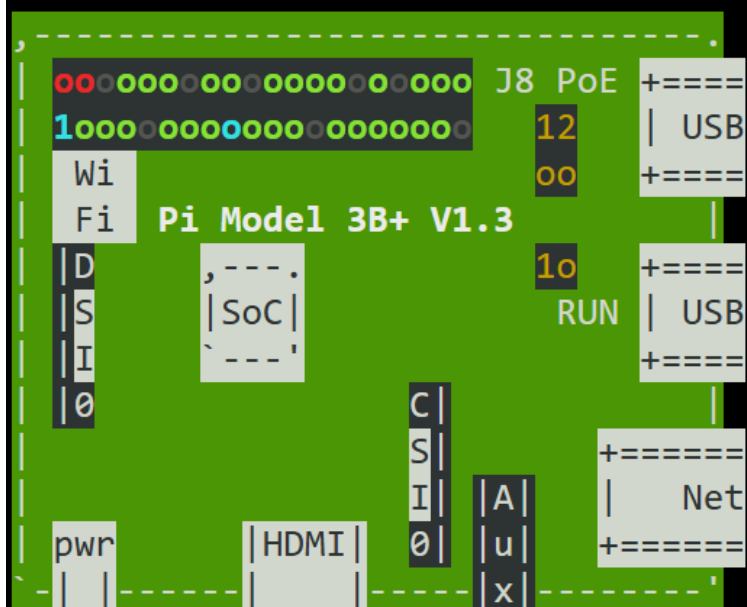


On your Raspberry Pi at the terminal window, type the command “pinout”. You should see something like the following:

```

Description      : Raspberry Pi 3B+ rev 1.3
Revision        : a020d3
SoC             : BCM2837
RAM             : 1GB
Storage         : MicroSD
USB ports       : 4 (of which 0 USB3)
Ethernet ports  : 1 (300Mbps max. speed)
Wi-fi          : True
Bluetooth       : True
Camera ports (CSI) : 1
Display ports (DSI): 1

```



3V3	(1)	(2)	5V
GPI02	(3)	(4)	5V
GPI03	(5)	(6)	GND
GPI04	(7)	(8)	GPI014
GND	(9)	(10)	GPI015
GPI017	(11)	(12)	GPI018
GPI027	(13)	(14)	GND
GPI022	(15)	(16)	GPI023
3V3	(17)	(18)	GPI024
GPI010	(19)	(20)	GND
GPI09	(21)	(22)	GPI025
GPI011	(23)	(24)	GPI08
GND	(25)	(26)	GPI07
GPI00	(27)	(28)	GPI01
GPI05	(29)	(30)	GND
GPI06	(31)	(32)	GPI012
GPI013	(33)	(34)	GND
GPI019	(35)	(36)	GPI016
GPI026	(37)	(38)	GPI020
GND	(39)	(40)	GPI021

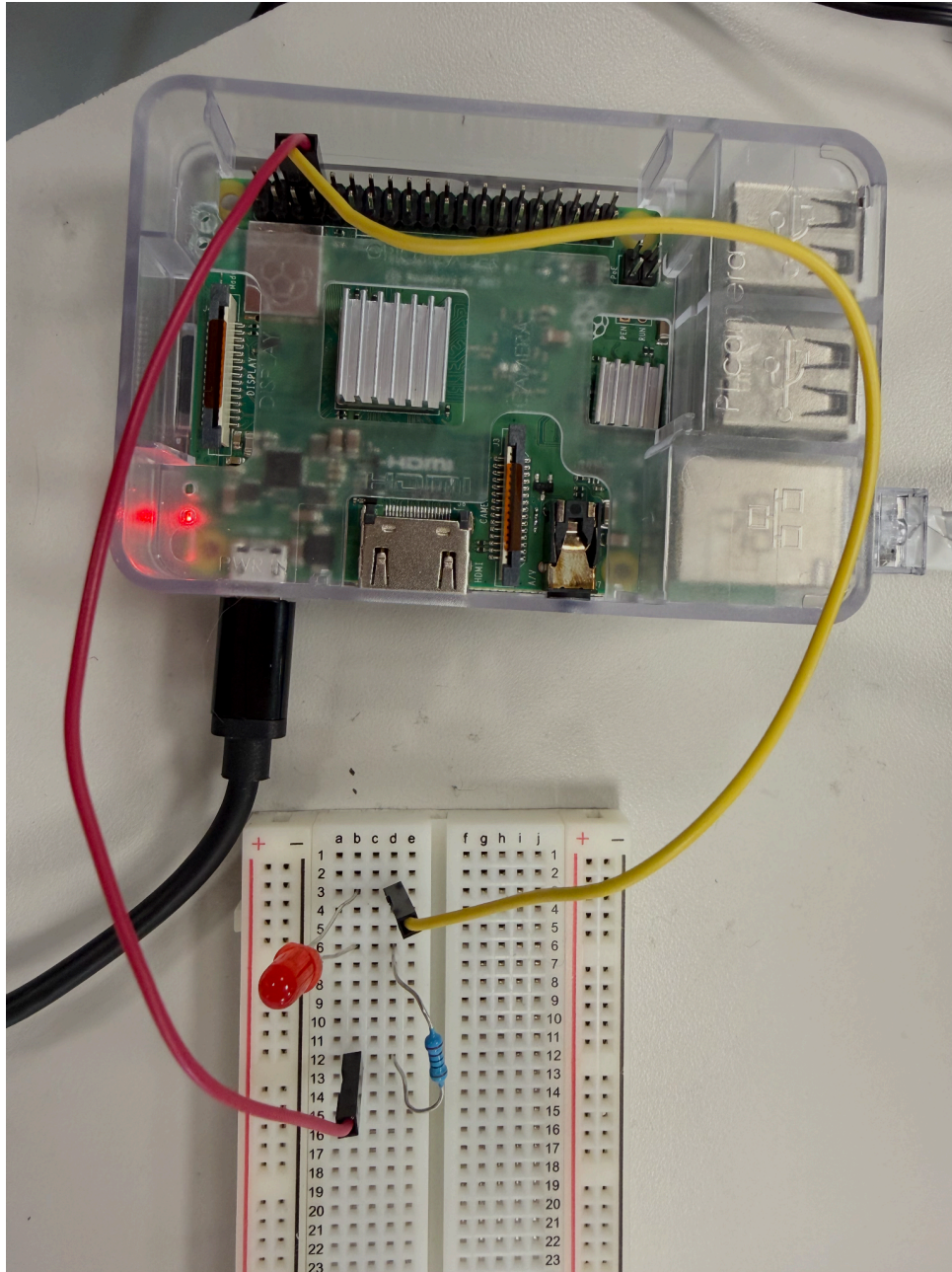
The top graphic shows the pins that are color coded and match the bottom description. The white numbers in parenthesis are the BOARD numbers, the GPIO numbers are the BCM descriptions. There are three types of pins: (1) ground pins: GND means “ground” – all circuits need to have a ground (2) power pins - 3V3 means 3.3 volt power, and 5V means 5 volts of power (3) GPIO which stands for “General Purpose Input/Output. These are the pins that we will be controlling with the gpiozero library. WARNING ABOUT GPIO PINS: These pins are fragile, if you accidentally plug 5V or more into a GPIO pin, you can fry the Pi.

To see an example of this set up the following circuit:

- The yellow wire is connecting BOARD pin 6, which is a ground pin to the negative terminal (the short side) of the LED.
- The LED positive leg (the long side) is connected to one side of the resistor.
- The other side of the resistor is connected to BOARD pin 8 which is GPIO14 in this case using the red wire.

NOTE that the color wire does not matter. Also note that the LED must be with the long side towards the power source and the short side towards ground. If you have your circuit set up correctly, if you connect the power side (the yellow wire that is connected to BOARD pin 8, you should be able to change this to BOARD pin 1 (the 3V3) and the LED should light up. If the LED does not light up, you have done something wrong! Here's a [video](#) that gives you more information about breadboards in general..

NOTE Avoid using GPIO0, GPIO1, GPIO2, GPIO3, GPIO14, GPIO15 for this project since they are special purpose. Any other GPIO pin should work.



To control the GPIO14 pin try the following program:

```
# gpiozero is a python package (library) we are only importing the code for the LED object
from gpiozero import LED
#time is another python package
from time import sleep

light = LED(14) #LED is an LED object from the library, "light" is the variable name

while True:
    light.on() #turn the light on (put power to the GPIO pin)
    sleep(1) #wait for 1 second before going to the next command
    light.off() #turn the light off (take away power from the GPIO pin)
    sleep(1) #wait for 1 second before going to the next command
```

At the command prompt, use python3 to run the program. Note that gpiozero needs python3 to run:

```
turtle@raspberrypi:~/Desktop/COSC1030/Hardware $ python3 blink_gpiozero.py
```

The light should be blinking.

Questions to answer:

1. How would you make the light blink faster? Slower? (Be sure to change the program to see if you are correct.)
2. How could you make the light stay on for twice as long as it is off? (Be sure to change the program to see if you are correct.)

Project: Traffic Light

Use this starter project to create a program that simulates a traffic light. The traffic light should repeat the sequence green for 5 seconds, yellow for 1 second, red for 3 seconds. Note that each LED should be connected to its own GPIO pin so it can be controlled by a separate circuit.

Hint: The code for the traffic light will all need to be in one `while True:` loop.

TURN IN: A video demonstration of your work. (Cell phone video is fine)